

Complexity Theory for Map Reduce

Geonmin Kim(204869)
Dept. of Software Engineering
Distributed Network System Laboratory
Chonnam National University



Index

1. Introduction
2. Reducer Size and Replication Rate
3. Similiarity Joins
4. A Graph Model for MapReduce Problems
5. Mapping Schemas
6. When Not All Inputs Are Presents
7. Lower Bounds on Replication Rate
8. Case Study : Matrix Multiplication

Index

1. Introduction

2. Reducer Size and Replication Rate

3. Similarity Joins

4. A Graph Model for MapReduce Problems

5. Mapping Schemas

6. When Not All Inputs Are Present

7. Lower Bounds on Replication Rate

8. Case Study : Matrix Multiplication

1. Introduction



This section derives into the trade - offs involved in designing efficient MapReduce Algorithms focusing on the communication cost between the Map and Reduce tasks.

- Communication as a Bottleneck

Communication between the Map and Reduce phases is often the primary bottleneck in MapReduce computations.

Reducing this cost is a critical factor in optimizing algorithm performance.

- Trade - offs with Other Goals

While minimizing communication is important, there are often trade - offs with other aspects.

- ① Wall - Clock time

The total time taken to execute the MapReduce job, which might increase if communication is reduced at the expense of parallelism or computational complexity.

- ② Main Memory Usage

Algorithms that reduce communication may require larger reducer sizes, meaning more memory is needed.

Index

1. Introduction
- 2. Reducer Size and Replication Rate**
3. Similarity Joins
4. A Graph Model for MapReduce Problems
5. Mapping Schemas
6. When Not All Inputs Are Presents
7. Lower Bounds on Replication Rate
8. Case Study : Matrix Multiplication

2. Reducer Size and Replication Rate



There are two parameters, 'Reducer Size' and 'Replication Rate'.

- **Parameter 1 : Reducer Size (Denoted by q)**

- Upper bound on the number of values that are allowed to appear in the list associated with single key.
- We can consider following two things when we choose the Reducer Size.

① To make Reducer Size small, we should have more number of Reducers.

To expect a low wall-clock time with a high degree of parallelism, we should create more Reduce tasks.

② Reducer Size must be sufficient enough to ensure that the computation is executed entirely in the main memory of its compute node.

- **Parameter 2 : Replication Rate (Denoted by r)**

- Average communication from (Map Tasks ~ Reduce Tasks) per input
- Measured by counting key-value pairs

2. Reducer Size and Replication Rate



ex) One-Pass Matrix Multiplication

- **Input Matrices ($n \times n$)**

- Matrix A with elements a_{ij}

- Matrix B with elements b_{jk}

- **Key - Value Pairs**

- Matrix A : n pairs, all with keys like (i,k) where k ranges from $1 \sim n$.

- Matrix B : n pairs, all with the same type of keys (i, k)

- **Reducer Size (q) and Replication Rate (R)**

- Reducer Size (q) : For each key (i, k) in Matrix A and B, we need $2n$ pairs in total.

- Replication Rate (r) : Average Number of key-value pairs produced for each element, which equals n .

- **Trade - Off between q and R**

- As we try to minimize the size of the q , r increases.

- $qr \geq (2n)^2$

Index

1. Introduction
2. Reducer Size and Replication Rate
- 3. Similarity Joins**
4. A Graph Model for MapReduce Problems
5. Mapping Schemas
6. When Not All Inputs Are Presents
7. Lower Bounds on Replication Rate
8. Case Study : Matrix Multiplication

3. Similarity Joins



- Similarity $s(x, y)$
 - In set X , we consider a similarity measure $s(x, y)$ that tells how similar two elements x and y of set X are.
 - Assume that similarity s is symmetric. ($s(x, y) = s(y, x)$)

- Output of the algorithm
 - Pairs whose similarity exceeds a given threshold α

- Assume that we have a large set of images (1TB, one million images each of size 1MB), and we want to find pairs of images that are similar based on the similarity $s(x, y)$.

- Input key-value pairs (i, P_i)
 - i is an ID for the picture, P_i is the picture itself.

3. Similarity Joins

- Initial approach : Let's compare every image with every other image by creating 'pairs'

- Pairs of $\{i, j\}$
- The input to the corresponding reducer will be $(\{i, j\}, [P_i, P_j])$.
- Reduce Function will then apply the similarity function s (compute $s(P_i, P_j)$).
- The Output will be the pairs of which each similarity exceeds threshold.

-> This approach is not appropriate because...

- Too high Replication Rate (999,999)
- A small reducer size (only 2MB)

->The total number of bytes communicated from tasks is

$1,000,000 \times 999,999 \times 1,000,000$

(pictures x the replication x the size of each pictures)

-> How to make a better approach? Let's make a 'group' of images!



3. Similarity Joins



- New Approach : **Group** the images into g groups, each containing $(10^6 / g)$ images.
 - Map Function
 - The input element : (i, P_i)
 - Generate $(g - 1)$ key-value pairs for each input element where the key is a pair of group IDs (u, v)
(Keys would be one of the sets $\{u, v\}$, u and v belongs to different group.)
 - Reduce Function
 - Takes each (i, P_i) and (j, P_j) where i and j belongs to different groups and applies the similarity function $s(P_i, P_j)$.
 - Each reducer processes images from two different groups of keys $\{u, v\}$.
 - For intra-group comparisons, we can do this in a **round - robin** manner across reducers
ensuring all pairs in a group are checked.
- ex) Compare the members of group u at the reducer $\{u, u+1\}$, where the “+1” is taken in the end-around sense.

3. Similarity Joins



- Result
 - Replication Rate (r) = $g - 1$ (approximately g)
 - Each input elements is turned into $(g - 1)$ key-value pairs.
 - Reducer Size (s) = $2 \times 10^6 / g$
 - Each reducer will handle about $(2 \times 10^6 / g)$ image data.
- > If g is 1000, the the input consumes about 2GB.
- > The total number of bytes communicated is now $(1,000,000 \times 999 \times 1,000,000)$
 - 1,000 times smaller than $1,000,000 \times 999,999 \times 1,000,000$ (Initial Approach)
 - Moreover we still have about half a million reducers. We can get as much parallelism as our computing cluster offers.

3. Similarity Joins

- When working with a large dataset, we might have many reducers to handle the comparisons but it's common to have fewer compute nodes than needed to run each reducer separately.
- Combining Reducers
 - To maximize efficiency, we can combine multiple reducers into a small number of tasks.
 - This helps us use our available compute resources more effectively, keeping all nodes busy.
- Computation Cost and Grouping
 - Computation Independence
 - : The overall cost of the algorithm doesn't depend on how many groups g we have.
 - Reason 1) Each reducer processes images that fit into the main memory.
 - Reason 2) The Similarity function s is applied to every unique pair of images only once, regardless of how the workload is distributed among the reducers.



Index

1. Introduction
2. Reducer Size and Replication Rate
3. Similiarity Joins
- 4. A Graph Model for MapReduce Problems**
5. Mapping Schemas
6. When Not All Inputs Are Presents
7. Lower Bounds on Replication Rate
8. Case Study : Matrix Multiplication

4. A Graph Model for MapReduce Problems

- Proving lower bounds on the replication rate as a function of reducer size
- Key idea : Reducers operate independently
 - > For each output, there must be some reducer that gets all of the inputs needed to compute that output
- Inputs : The elements or data points you start with (e.g., images or matrix elements)
- Outputs : The results you aim to generate (e.g., pairs of similar images or products of matrix elements)
- Again, each output must have a specific reducer that receives all required inputs.



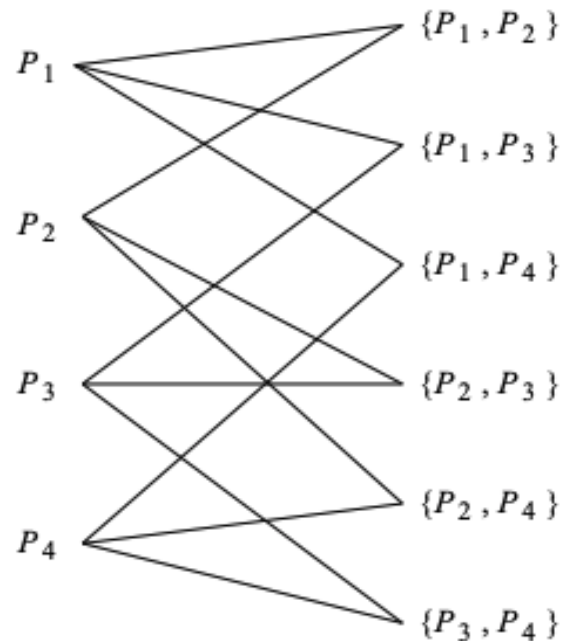
4. A Graph Model for MapReduce Problems

- Similarity Join (Example 2.19) -> all - pairs problem



If you have 4 images, the graph shows the possible pairs of images as outputs.

There are 6 pairs and each output is connected to its corresponding input images.



-> This could be considered as a Combination calculation $\binom{n}{2} = \frac{n!}{2!(n-2)!}$

4. A Graph Model for MapReduce Problems

- Matrix Multiplication (Example 2.20)

If we multiply two $n \times n$ matrices (M and N) to get matrix P, you have $2n^2$ inputs (= individual matrix elements m_{ij} , n_{jk}) and n^2 outputs (p_{ik}).

Each output p_{ik} is related to $2n$ inputs : $m_{i1}, m_{i2}, m_{i3} \dots m_{in} // n_{1k}, n_{2k}, n_{3k} \dots n_{nk}$

Example) In 2×2 matrices,

-> 8 inputs, 4 outputs and each output is related to 4 inputs.

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} e & f \\ g & h \end{bmatrix} = \begin{bmatrix} i & j \\ k & l \end{bmatrix}$$

Each output depends on specific inputs, highlighting a complex many - to - many relationship.



4. A Graph Model for MapReduce Problems



Q. What if inputs and/or outputs not all be present in any instance of problem?

- Handling Missing Inputs / Outputs

In the natural join of two tables $R(A, B)$ and $S(B, C)$:

- Inputs : All possible combinations of tuples from R and S based on attributes
- Outputs : All possible triples of A, B and C which would be formed like (a, b, c)

When performing the join, not all potential R -tuples or S -tuples may be present and as a result, you might only produce a **subset** of the possible output triples (a, b, c) .

But it is important to know the fact that the graph model **still represents all possible relationships**.

This means,

the connections between inputs and outputs are defined broadly, **regardless of what is actually present** in a particular instance.

Index

1. Introduction
2. Reducer Size and Replication Rate
3. Similiarity Joins
4. A Graph Model for MapReduce Problems
- 5. Mapping Schemas**
6. When Not All Inputs Are Presents
7. Lower Bounds on Replication Rate
8. Case Study : Matrix Multiplication

5. Mapping Schema

When we represent MapReduce problems as graphs, we should define MapReduce algorithms to solve problems.



Each such MapReduce algorithm must have a mapping schema.

- Mapping Schema

- expresses how outputs are produced by the various reducers used by the algorithm.

In case of a mapping schema for a given problem with a given Reducer Size q ,

- ① Reducer Size Constraint

- **No Reducer Overload** : Each reducer can only handle a limited number of inputs defined as q .

- This is a fundamental aspect of the reducer's capacity and helps maintain efficiency and manageability during processing.

- ② Coverage of Outputs

- **Output Production Requirement** : For every output that aims to produce, there must be at least one reducer assigned all the necessary inputs to that output. -> We say this 'the reducer covering the output'

If a reducer does not have access to all the inputs required for an output,
it cannot correctly compute that output than the algorithm fails.

5. Mapping Schema



Example 2.21

▪ Grouping Strategy Overview

- Input and Grouping

Suppose you have p pictures and these pictures are divided into g equal - sized groups (p / g pictures per group).

- Output

The total number of outputs : $\binom{p}{2}$ which is approximately $\frac{p^2}{2}$

- Reducer Configuration

Each reducer will handle inputs from two groups ($2p / g$ inputs). -> Therefore, the required Reducer Size q is $2p / g$

- Replication Rate

Indicating how many times each picture is replicated across reducers.

-> Replication Rate R is apporximately $g - 1$ (simply g).

- Inversely Proportional Relationship

- When we express r in terms of q , we can get $r = 2p / q$

-> When Reducer Size q decreaes, the Replication Rate r increases.

5. Mapping Schema



Example 2.21

▪ Mapping Schema

- Number of Reducers $\frac{g^2}{2}$

Each reducer corresponds to a pair of groups.

Thus, no reducer is assigned more than $2p / g$ inputs ensuring that Reducer Size constraint is satisfied.

- Every Output (pair of pictures) is covered by a reducer:

- ① If the output is a pair from two different groups u and v , the reducer for the pair $\{u, v\}$ handles it.
- ② If the output involves only one group u , several reducers corresponding to pairs $\{u, v\}$ can compute that output.
- ③ The transformation of inputs into outputs can occur not only in Map phase but also in Reduce phase.

▪ Key - Value Pairs

- Each input generates at least one key - value pair used to compute its related output.

- The fact that an output depends on a certain input means that when that input is processed at the Map task, there will be at least one key -value pair generated to be used when computing that output.

Index

1. Introduction
2. Reducer Size and Replication Rate
3. Similiarity Joins
4. A Graph Model for MapReduce Problems
5. Mapping Schemas
- 6. When Not All Inputs Are Presents**
7. Lower Bounds on Replication Rate
8. Case Study : Matrix Multiplication

6. When Not All Inputs Are Present



- An algorithm dealing with potential missing outputs still requires a mapping schema to ensure it can produce all possible outputs if the related inputs exist.

- The input set may not be fully present.

=> The absence of some inputs affects the choice of Reducer Size (q).

If q is set to fit the inputs into memory, it may need to be adjusted upwards, accounting for the fact that some inputs might not be present.

- Example 2.22

Assume that the Reduce Function can handle a key and its associated list of q values in memory.

And only 5% of the possible inputs are present in the dataset.

-> Each reducer will receive about $q/20$ of the existing inputs.

-> The algorithm can use a reducer size of $20q$, so that, on average, each reducer will receive around q inputs.

+ Regarding the randomness in how inputs are distributed, it is suggested to choose a slightly smaller reducer size to ensure efficient processing.

Index

1. Introduction
2. Reducer Size and Replication Rate
3. Similiarity Joins
4. A Graph Model for MapReduce Problems
5. Mapping Schemas
6. When Not All Inputs Are Presents
- 7. Lower Bounds on Replication Rate**
8. Case Study : Matrix Multiplication

7. Lower Bounds on Replication Rate



- Technique to prove a lower bound on communication for similarity - join algorithms.
- Goal : Ensure optimal trade - offs between communication, reducer size, and parallelism while minimizing communication costs.

① Upper Bound

Prove an upper bound on the number of outputs a reducer with q inputs can handle, denoted as $g(q)$

② Total Outputs

Determine the total number of outputs produced by the problem

③ Inequality

If there are k reducers with $q_i < q$ inputs for each reducer,

then the sum of $g(q_i)$ across all reducers must be at least as large as the total number of outputs.

④ Lower Bound

Manipulate inequality (3) to get a lower bound on the sum of q_i (the total of inputs across reducers).

This often involves replacing some q_i factors with the upper bound q .

⑤ Replication Rate

Since the sum of q_i represents the total communication, divide this lower bound (4) by the number of inputs to get a lower bound on replication rate, which is the key metric for communication efficiency.

7. Lower Bounds on Replication Rate



▪ Example 2.23

How to derive a lower bound on the replication rate r for the all - pairs problem

① Upper Bound on Reducer Outputs

A reducer with q inputs can cover at most $\binom{q}{2}$, approximately $(q^2)/2$ outputs.

② Total Outputs

Approximately $(p^2)/2$, where p is the number of inputs.

③ Constructing the Inequality

The inequality becomes $\sum_{i=1}^k q_i^2/2 \geq p^2/2$, which simplifies to $\sum_{i=1}^k q_i^2 \geq p^2$.

④ Manipulating the Inequality

Replace one factor of q_i with q (where $q \geq q_i$), leading to $q \sum_{i=1}^k q_i \geq p^2$

⑤ Replication Rate

Divide both sides by p , giving the lower bound $r \geq (p/q)$

=> This proves that the Replication Rate r is at least p/q ,

meaning the family of algorithms discussed in Example 2.21 has a Replication Rate that is at most twice the minimum possible value.

Index

1. Introduction
2. Reducer Size and Replication Rate
3. Similarity Joins
4. A Graph Model for MapReduce Problems
5. Mapping Schemas
6. When Not All Inputs Are Presents
7. Lower Bounds on Replication Rate
- 8. Case Study : Matrix Multiplication**

8. Case Study: Matrix Multiplication

- Applying the lower-bound technique to one - pass matrix multiplication algorithms

We can group rows and columns of the two input matrices into bands.

Each pair consisted of a band of rows of the first matrix and a band of columns of the second matrix is used by one reducer to produce a square elements of the output matrix.

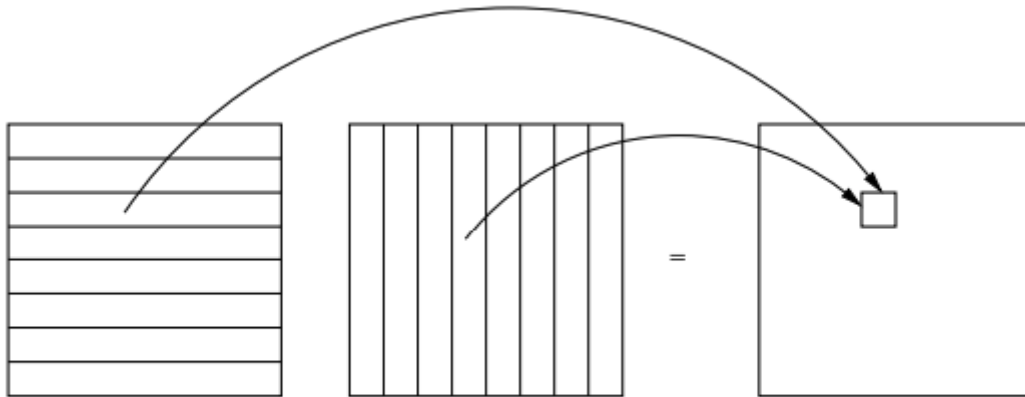


Figure 2.11: Dividing matrices into bands to reduce communication

Suppose, we want to compute $MN = P$ and all three matrices are $n \times n$.

For Group g ,

rows of the matrix $M \Rightarrow$ group in to n/g rows each

columns of matrix $N \Rightarrow$ group into n/g columns each



8. Case Study: Matrix Multiplication

- **One - Pass Algorithm**
- Map Function

For each element of M, the Map Function generates g key - value pairs

(The value is the element itself and the key is the group to which the element belongs paired with any of the groups of the N)

For each element of N, the Map Function generates g key - value pairs

(The value is the element itself + its row and column and the key is the group of that element paired with any of the groups of M)

- Reduce Function (corresponding to the key (i, j) where i belongs to M and j belong to N)

Reduce Function gets a value list consisting of all the elements in the i th band of M and the j th band of N.



8. Case Study: Matrix Multiplication

- One - Pass Algorithm

Now, each elements of M and N is mapped to g reducers.

Each reducer gets $n(n/g)$ elements from each two matrices, so $q = 2n^2/g$ elements in total and produces output for the corresponding square of P.

- Replication Rate : g (how many times each element is sent to reducers)

Combining $r = g$ with $q = 2n^2/g$, we can conclude that

- Communication Cost : $r = 2n^2/q$ (inversely proportional to the Reducer Size q)

This establishes a lower bound on communication for the one - pass approach, showing that $r = 2n^2/q$.

It turns out that this upper bound on Replication Rate is also a lower bound.

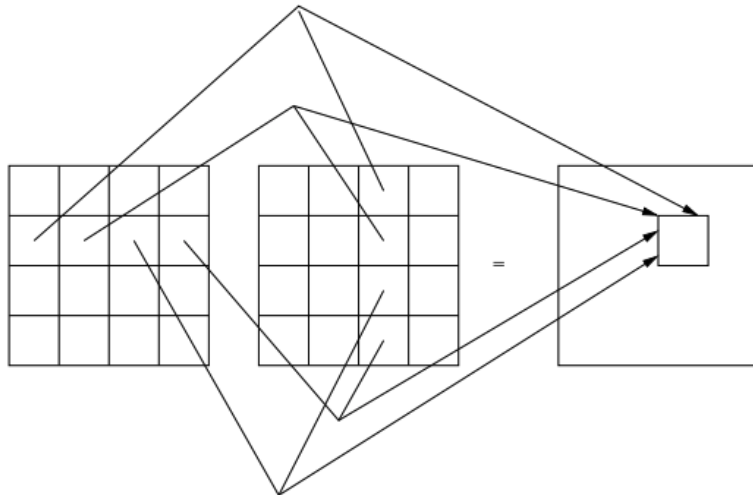
This means the communication cannot be improved further for the one - pass method.



8. Case Study: Matrix Multiplication

■ Two - Pass Algorithm

- The two - pass matrix multiplication algorithms offers a way to reduce total communication.
- The reducer would get two elements m_{ij} and n_{jk} .
- We may divide the rows and columns of both input matrices M and N into g groups (n / g rows or columns each).
- Both matrices are divided into g^2 smaller squares (of n^2/g^2 elements each),
with each square of M and N processed together to compute partial contributions to the corresponding square in P .



8. Case Study: Matrix Multiplication



- **Two - Pass Algorithm**

- In First pass, the pairs of squares from M and N are multiplied for all (I, J, K).
- In Second pass, for each I and K we sum the products over all possible sets J.

- **Map Function**

The Map function generates g key - value pairs with value consisting of n_{jk} , j , and k , and with keys (I, J, K) for any group I.

- **Reduce Function**

The reducer corresponding to (I, J, K) receives as input all the elements m_{ij} and n_{jk} and it computes $x_{ijk} = \sum_{j \text{ in } J} m_{ij} n_{jk}$.

Since,

- Replication Rate : g
- Total Communication Cost : $2gn^2$
- Reducer Size : $q = 2n^2/g^2$
- Total Communication for $2gn^2$: $2\sqrt{2}n^3/\sqrt{q}$

8. Case Study: Matrix Multiplication



- Two - Pass Algorithm (Second MapReduce Job)

In second MapReduce job, it sums up the x_{ijk} 's over all sets J .

- Map Function

The Map Function takes as input one element x_{ijk} , which we assume the previous reducers have left labeled with i and k .

-> One key - value pair is generated. (Key : (i, k) / Value : x_{ijk})

- Reduce Function

The Reduce Function simply sums the values associated with key (i, k) to compute the output element P_{ik} .

Regarding $q = 2n^2/g^2$ in the first MapReduce job, we can write $g = n\sqrt{2/q}$,

then the communication for the second job is $n^2g = \sqrt{2}n^3/\sqrt{q}$. (multiply n^2 on both sides)

- Total Communication for the second job

: $\sqrt{2}n^3/\sqrt{q}$ (Exactly half the communication for the first job)

- The total communication for the two - pass algorithm

: $3\sqrt{2}n^3/\sqrt{q}$. ($2\sqrt{2}n^3/\sqrt{q} + \sqrt{2}n^3/\sqrt{q}$)

8. Case Study: Matrix Multiplication

- Comparison between one - pass and two - pass algorithms

Communication cost for the one - pass algorithm : $4n^4/q$.

Communication cost for the two - pass algorithm : $3\sqrt{2}n^3/\sqrt{q}$.

- Both algorithms do the same $O(n^3)$

$$n^4/q > n^3/\sqrt{q}$$

If q is close to its minimum possible value of $2n$, then the two - pass algorithm beats one - pass algorithm by a factor of $O(\sqrt{n})$



Thank You